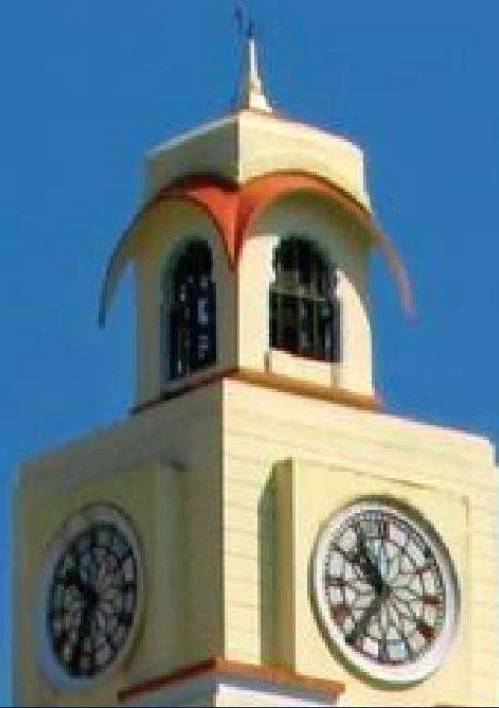




BITS Pilani
Pilani Campus

Support Vector Machines

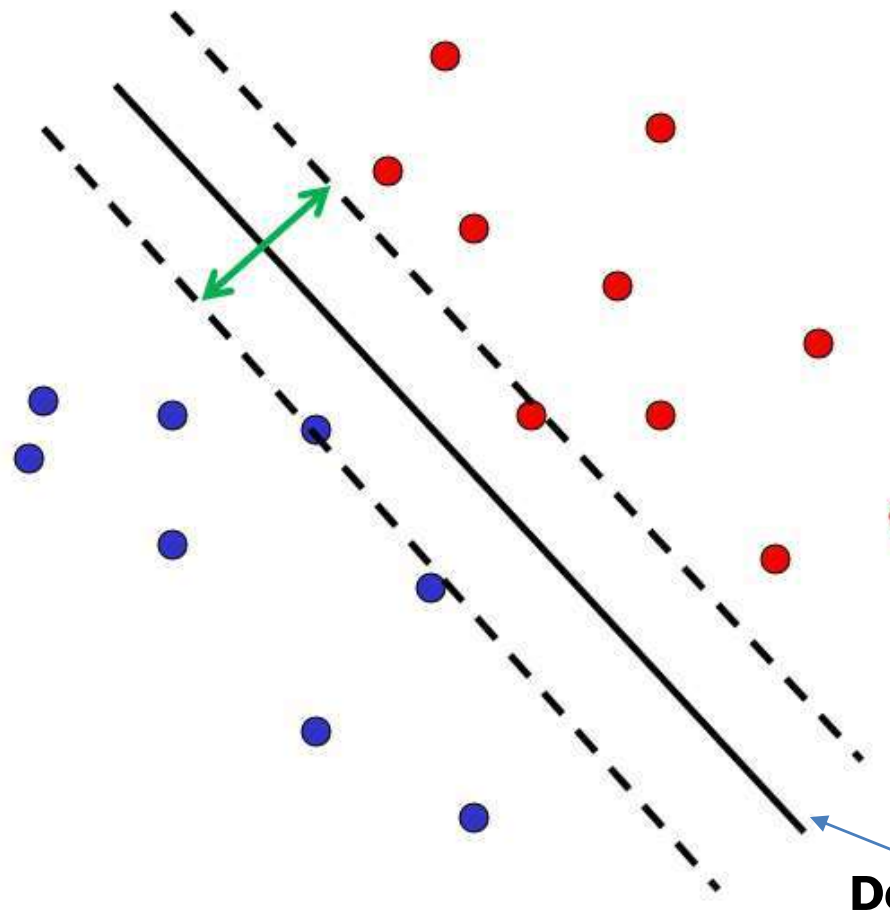
MFML Team

Topics to be covered



- ▶ Last class we saw how to deal with hard and soft margin SVM problems
- ▶ In this class we will see how to deal with non linear SVM (Kernel Trick)
- ▶ We will go through one nonlinear example: XOR problem
- ▶ We will see Examples for soft margin SVM using the slack variables (Covered in the last lecture)
- ▶ We will see Examples for soft margin SVM using hinge loss function (Covered in last lecture)

Linear Classifier



- Discriminative classifier based on *optimal separating line (for 2d case)*
- Maximize the *margin* between the positive and negative training examples

C. Burges, [A Tutorial on Support Vector Machines for Pattern Recognition](#), Data Mining and Knowledge Discovery, 1998

Maximum Margin

• denotes +1

○ denotes -1

Support Vectors

are those datapoints that the margin pushes up against

1. If hyperplane is oriented such that it is close to some of the points in your training set, new data may lie on the wrong side of the hyperplane, even if the new points lie close to training examples of the correct class.
2. Solution is maximizing the margin

with the,
maximum margin.

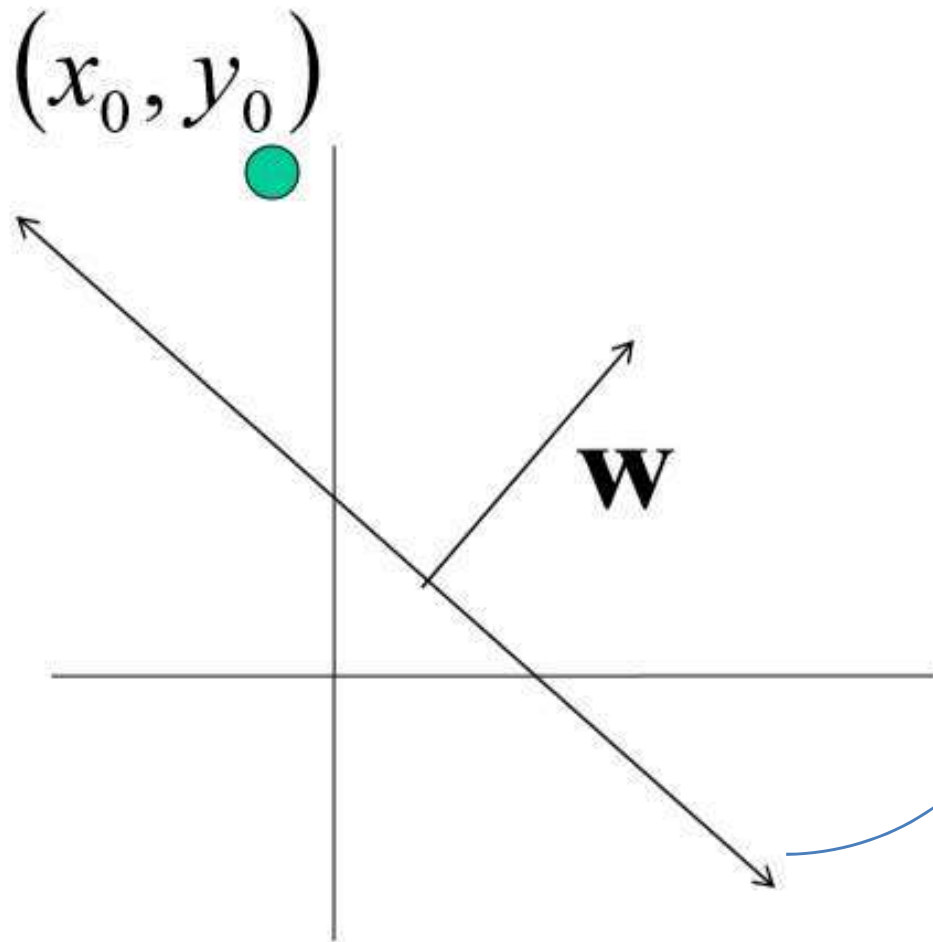
This is the
simplest kind of
SVM (Called an
LSVM)

Linear SVM

Support Vectors

- Geometric description of SVM is that the max-margin hyperplane is completely determined by those points that lie nearest to it.
- Points that lie on this margin are the support vectors.
- The points of our data set which if removed, would alter the position of the dividing hyperplane

Line with 2 features: R2



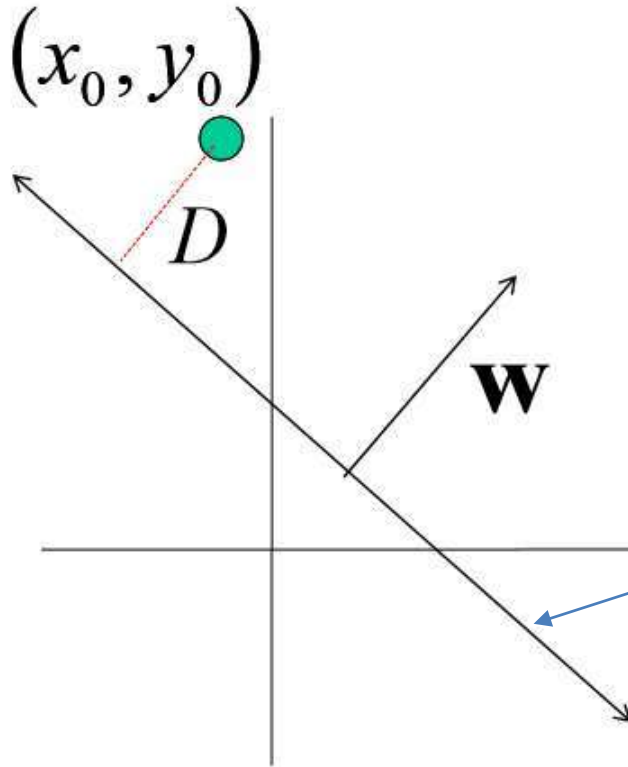
Let $\mathbf{w} = \begin{bmatrix} a \\ c \end{bmatrix}$ $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$

$$ax + cy + b = 0$$



$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

Line with 2 features: R2



Let $\mathbf{w} = \begin{bmatrix} a \\ c \end{bmatrix}$ $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$

$$ax + cy + b = 0$$



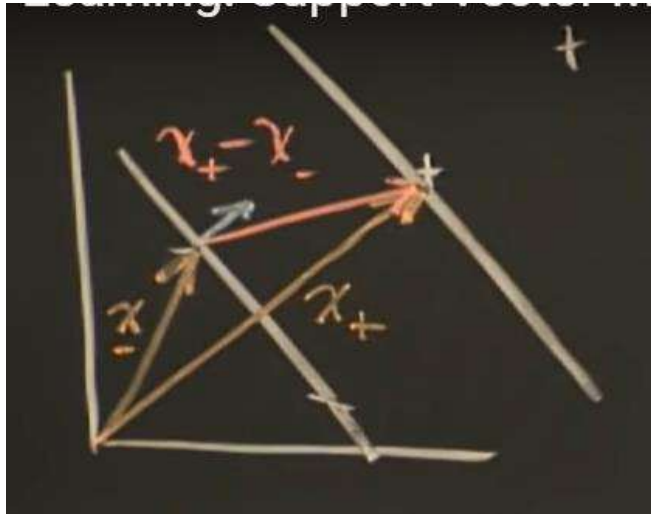
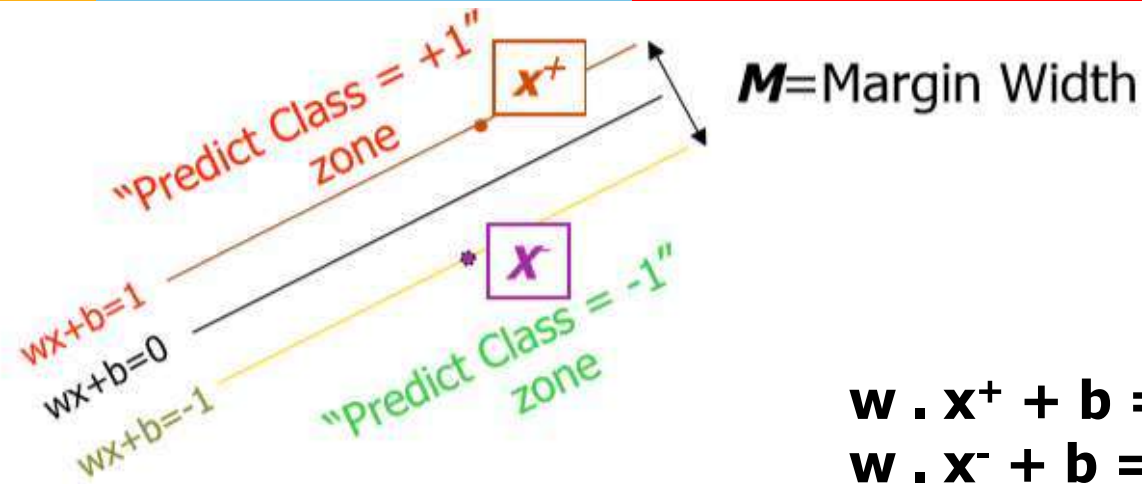
$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

$$D = \frac{|ax_0 + cy_0 + b|}{\sqrt{a^2 + c^2}} = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|} \quad \left. \vphantom{\frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}} \right\} \begin{array}{l} \text{distance from} \\ \text{point to line} \end{array}$$

Kristen Grauman

<https://brilliant.org/wiki/dot-product-distance-between-point-and-a-line/>

Linear SVM Mathematically



$$w \cdot x^+ + b = +1$$

$$w \cdot x^- + b = -1$$

Margin width

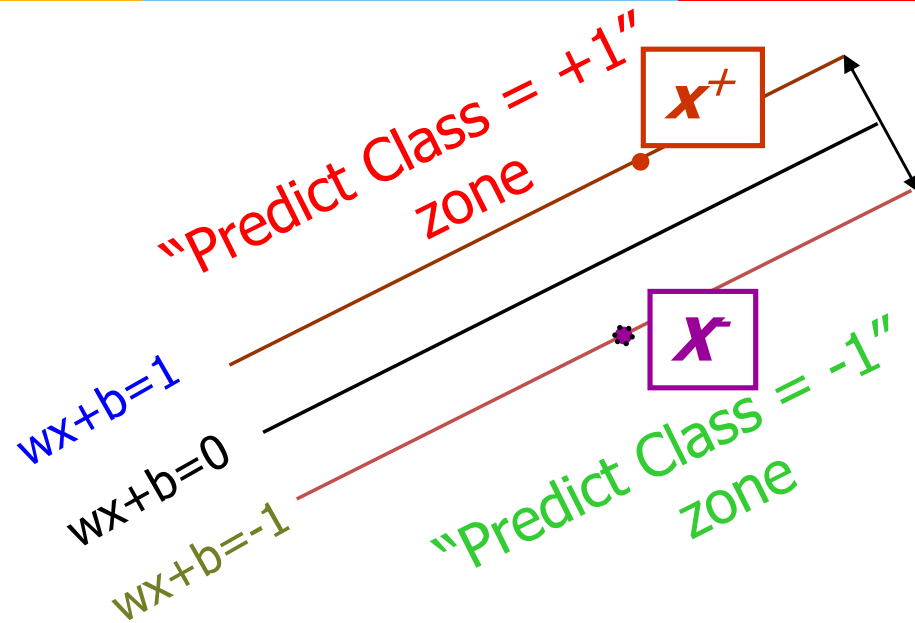
$$= x^+ - x^-, \frac{w}{||w||}$$

$$= \frac{w \cdot x^+ - w \cdot x^-}{||w||}$$

$$= (1-b) - (-1-b) / ||w||$$

$$= \frac{2}{||w||}$$

Linear SVM Mathematically



M=Margin Width

Distance between lines given by solving linear equation:

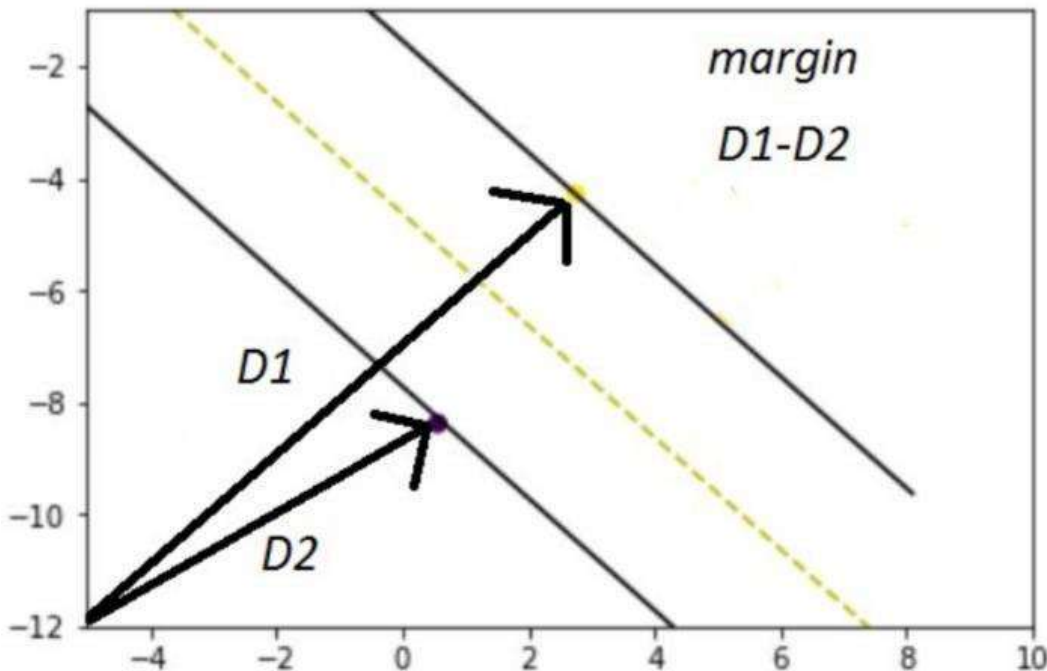
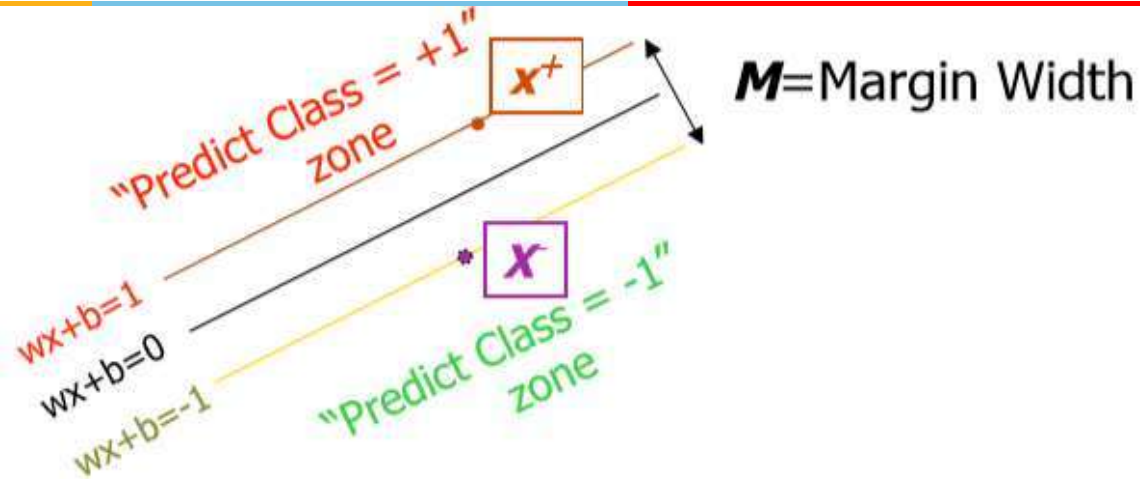
What we know:

- $w \cdot x^+ + b = +1$
- $w \cdot x^- + b = -1$

Maximize margin: $M = \frac{2}{||w||}$

Equivalent to minimize: $\frac{1}{2} ||w||^2$

Linear SVM Mathematically



$$D1 = w^T x + b = 1 \quad w^T x + b - 1 = 0$$

$$D2 = w^T x + b = -1 \quad w^T x + b + 1 = 0$$

$$w^T x + b - 1 - (w^T x + b + 1)$$



Solve algebraically

$$\frac{2}{|w|}$$

Solving the Optimization Problem

1. Maximize margin $2/\|\mathbf{w}\|$
2. Correctly classify all training data points:

$$\mathbf{x}_i \text{ positive } (y_i = 1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 1$$

$$\mathbf{x}_i \text{ negative } (y_i = -1): \quad \mathbf{x}_i \cdot \mathbf{w} + b \leq -1$$

Quadratic optimization problem:

Find $\mathbf{w}$ and b such that

$\Phi(\mathbf{w}) = \frac{1}{2}\|\mathbf{w}\|^2$ is minimized;

and for all $\{(\mathbf{x}_i, y_i)\}: y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$

$$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

$$+1(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

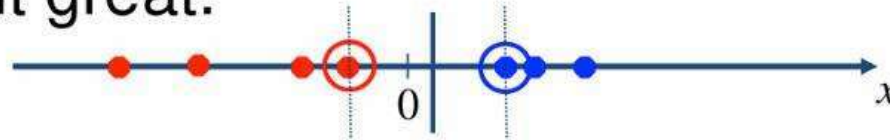
$$-1(\mathbf{w}^T \mathbf{x}_i + b) \leq 1$$

$$\text{same as } (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

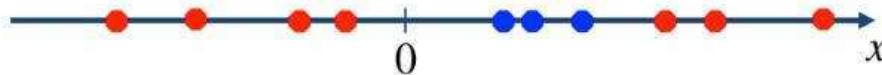
Nonlinear SVM



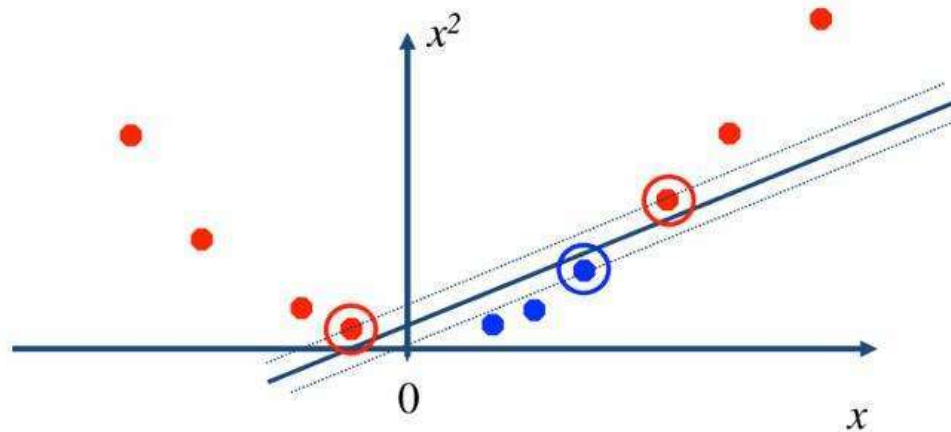
- Datasets that are linearly separable with some noise soft margin work out great:



- But what are we going to do if the dataset is just too hard?



- How about... mapping data to a higher-dimensional space:



Kernel Functions



- ▶ A kernel function is some function that corresponds to an inner product in some expanded feature space.

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

- ▶ Linear classifier relies on dot product between vectors $x_i^T x_j$
- ▶ If every data point is mapped into high-dimensional space via some transformation $\phi : x \rightarrow \phi(x)$, the dot product becomes:
 $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$
- ▶ For some functions $K(x_i, x_j)$ checking $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ is difficult.
- ▶ Mercer's theorem: Every positive-semidefinite symmetric function is a kernel function.



1) We can *construct kernels from scratch*:

- For any $\varphi : \mathcal{X} \rightarrow \mathbb{R}^m$, $k(x, x') = \langle \varphi(x), \varphi(x') \rangle_{\mathbb{R}^m}$ is a kernel.
- If $d : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is a *distance function*, i.e.
 - $d(x, x') \geq 0$ for all $x, x' \in \mathcal{X}$,
 - $d(x, x') = 0$ only for $x = x'$,
 - $d(x, x') = d(x', x)$ for all $x, x' \in \mathcal{X}$,
 - $d(x, x') \leq d(x, x'') + d(x'', x')$ for all $x, x', x'' \in \mathcal{X}$,

then $k(x, x') := \exp(-d(x, x'))$ is a kernel.

Kernel Functions Construction



2) We can *construct kernels from other kernels*:

- if k is a kernel and $\alpha > 0$, then αk and $k + \alpha$ are kernels.
- if k_1, k_2 are kernels, then $k_1 + k_2$ and $k_1 \cdot k_2$ are kernels.

Examples of Kernels

- ▶ Linear: $K(x_i, x_j) = x_i^T x_j$
- ▶ Polynomial of power p : $K(x_i, x_j) = (1 + x_i^T x_j)^p$
- ▶ Sigmoid: $K(x_i, x_j) = \tanh(\beta_0 x_i^T x_j + \beta_1)$

The Kernel Trick



- The linear classifier relies on dot product between vectors
 - $\mathbf{x}_i^T \cdot \mathbf{x}_j$
- If every data point is mapped into high-dimensional space via some transformation Φ : $\mathbf{x} \rightarrow \phi(\mathbf{x})$, the dot product becomes:
$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$
- A *kernel function* is some function that corresponds to an inner product in some expanded feature space.

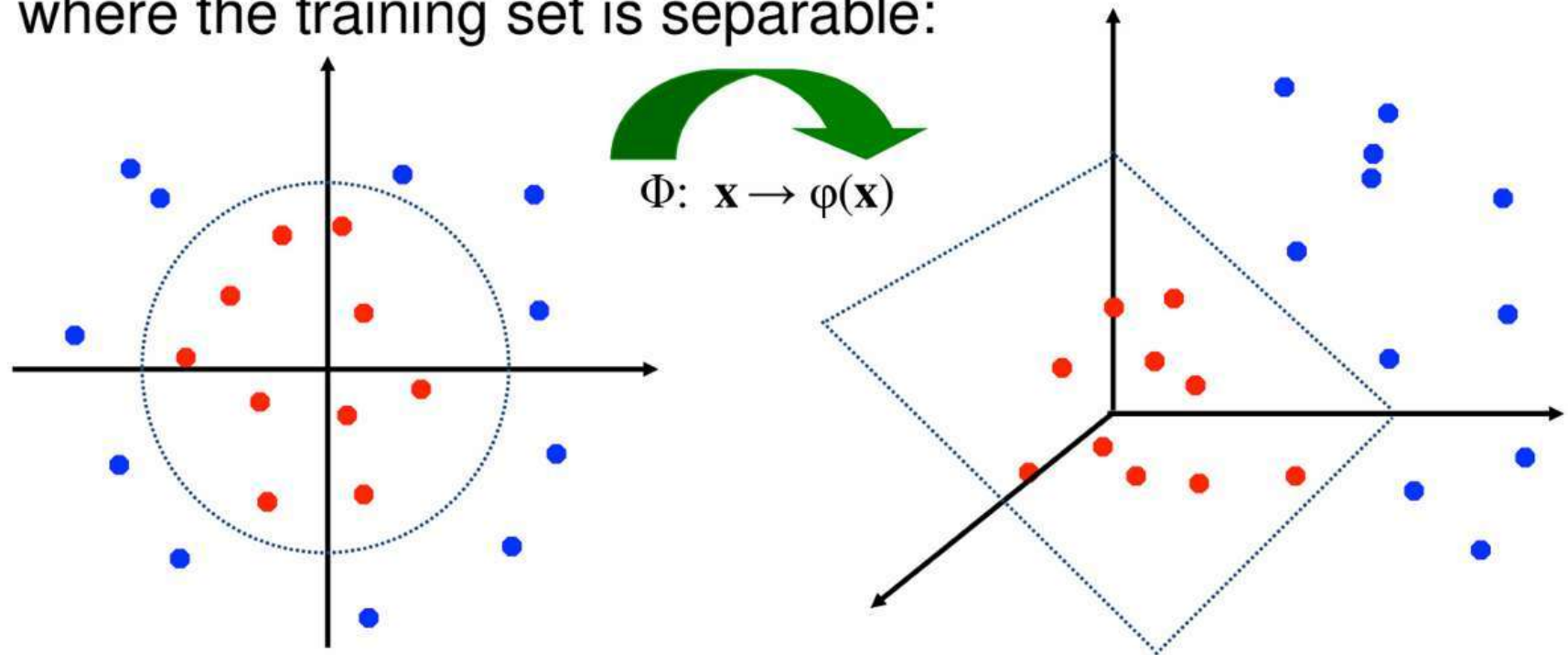


- SVM algorithms use a set of mathematical functions that are defined as the kernel.
- Function of kernel is to take data as input and transform it into the required form.
- Different SVM algorithms use different types of kernel functions. Example *linear, nonlinear, polynomial, and sigmoid etc.*

Nonlinear SVM – Feature Spaces



- General idea: the original input space can always be mapped to some higher-dimensional feature space where the training set is separable:



Steps to Solve Nonlinear SVM



1. **Select a kernel function.**
2. **Compute pairwise kernel values** between labelled examples.
3. Use this “**kernel matrix**” to solve for SVM support vectors and α weights.
4. To **classify a new example**: compute kernel values between the new input and support vectors, apply α weights, check the sign of output.

Nonlinear SVM – XOR Problem



For the XOR problem, the dataset is:

Point	x_1	x_2	Label y
$\mathbf{x}_1$	-1	-1	-1
$\mathbf{x}_2$	-1	$+1$	$+1$
$\mathbf{x}_3$	$+1$	-1	$+1$
$\mathbf{x}_4$	$+1$	$+1$	-1

Choose the Kernel for XOR Problem



Let us consider the XOR problem using SVMs (*Cherkassky, 1998*).

Start with a **kernel**:

$$K(\mathbf{z}, \mathbf{z}_j) = (1 + \mathbf{z}^\top \mathbf{z}_j)^2, \quad \mathbf{z} = [z_1, z_2]^\top \quad \mathbf{z}_j = [x_{j1}, x_{j2}]^\top.$$

Expressing the Kernel as Inner Product of ϕ Functions



$$\begin{aligned} K(\mathbf{z}, \mathbf{z}_j) &= (1 + \mathbf{z}^\top \mathbf{z}_j)^2 \\ &= (1 + x_{j1}z_1 + x_{j2}z_2)^2 \\ &= 1 + x_{j1}^2 z_1^2 + x_{j2}^2 z_2^2 + 2x_{j1}z_1 + 2x_{j2}z_2 + 2x_{j1}x_{j2}z_1z_2. \end{aligned}$$

Let us express $K(,) = \langle \phi(\mathbf{z}), \phi(\mathbf{z}_j) \rangle$, where:

$$\phi(\mathbf{z}) = [1 \quad z_1^2 \quad \sqrt{2} z_1 z_2 \quad z_2^2 \quad \sqrt{2} z_1 \quad \sqrt{2} z_2]^\top$$

$$\phi(\mathbf{z}_j) = [1 \quad x_{j1}^2 \quad \sqrt{2} x_{j1} x_{j2} \quad x_{j2}^2 \quad \sqrt{2} x_{j1} \quad \sqrt{2} x_{j2}]^\top$$

Computing the Kernel Matrix



For the XOR problem, compute $K(\mathbf{z}, \mathbf{z}_j) = (1 + \mathbf{z}^\top \mathbf{z}_j)^2$ for each pair:

	x_1	x_2	y
$\mathbf{x}_1$	-1	-1	-1
$\mathbf{x}_2$	-1	+1	+1
$\mathbf{x}_3$	+1	-1	+1
$\mathbf{x}_4$	+1	+1	-1

$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$, giving the kernel matrix:

$$K = \begin{bmatrix} 9 & 1 & 1 & 1 \\ 1 & 9 & 1 & 1 \\ 1 & 1 & 9 & 1 \\ 1 & 1 & 1 & 9 \end{bmatrix}$$

The Objective Function in the Dual Problem



From the dual problem, the objective $Q(\alpha)$ is:

$$\begin{aligned} Q(\alpha) &= \sum_{i=1}^4 \alpha_i - \frac{1}{2} \sum_{j=1}^4 \sum_{i=1}^4 \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \\ &= \alpha_1 + \alpha_2 + \alpha_3 + \alpha_4 \\ &\quad - \frac{1}{2} (9\alpha_1^2 - 2\alpha_1\alpha_2 - 2\alpha_1\alpha_3 + 2\alpha_1\alpha_4 \\ &\quad + 9\alpha_2^2 + 2\alpha_2\alpha_3 - 2\alpha_2\alpha_4 + 9\alpha_3^2 - 2\alpha_3\alpha_4 + 9\alpha_4^2) \end{aligned}$$

Solving for α s



Setting $\frac{\partial Q(\alpha)}{\partial \alpha_i} = 0$ for $i = 1, \dots, 4$ and using the constraint $\sum_i \alpha_i y_i = 0$, we obtain the system:

$$\begin{cases} 9\alpha_1 - \alpha_2 - \alpha_3 + \alpha_4 = 1 \\ -\alpha_1 + 9\alpha_2 + \alpha_3 - \alpha_4 = 1 \\ -\alpha_1 + \alpha_2 + 9\alpha_3 - \alpha_4 = 1 \\ \alpha_1 - \alpha_2 - \alpha_3 + 9\alpha_4 = 1 \end{cases}$$

Solving this set of equations yields:

$$\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \frac{1}{8}.$$

Hence all four points are **support vectors**.

Computing $\mathbf{w}_0$



Now, all 4 input points $\{\mathbf{z}_i\}_{i=1}^4$ are support vectors.

$$Q(\alpha) = \frac{1}{4}, \quad \frac{1}{2} \|\mathbf{w}_0\|^2 = \frac{1}{4} \Rightarrow \|\mathbf{w}_0\| = \frac{1}{\sqrt{2}}.$$

$$\mathbf{w}_0 = \sum_{i=1}^4 \alpha_i y_i \phi(\mathbf{z}_i) = \frac{1}{8} \left[-\phi(\mathbf{z}_1) + \phi(\mathbf{z}_2) + \phi(\mathbf{z}_3) - \phi(\mathbf{z}_4) \right]$$

Evaluating the ϕ components (noting cancellations):

$$\mathbf{w}_0 = \left[0, 0, -\frac{1}{\sqrt{2}}, 0, 0, 0 \right]^\top.$$

Computing b



For the support vectors we have:

$$\sum_{i=1}^4 \alpha_i y_i K(\mathbf{z}_i, \mathbf{x}_s) + b = +1 \quad (\mathbf{x}_s \text{ is any support vector})$$

Pick $\mathbf{x}_s = \mathbf{z}_2$ or $\mathbf{z}_4$. Using

$$K = \begin{bmatrix} 9 & 1 & 1 & 1 \\ 1 & 9 & 1 & 1 \\ 1 & 1 & 9 & 1 \\ 1 & 1 & 1 & 9 \end{bmatrix},$$

and $\alpha_i = \frac{1}{8}$

$$\frac{-K(\mathbf{z}_1, \mathbf{z}_2)}{8} + \frac{K(\mathbf{z}_2, \mathbf{z}_2)}{8} + \frac{K(\mathbf{z}_3, \mathbf{z}_2)}{8} - \frac{K(\mathbf{z}_4, \mathbf{z}_2)}{8} + b = 1$$

$$\Rightarrow -\frac{1}{8} + \frac{9}{8} + \frac{1}{8} - \frac{1}{8} + b = 1 \Rightarrow \boxed{b = 0}.$$

Decision Boundary for XOR Problem



The optimal hyperplane is given by:

$$\mathbf{w}_0^\top \phi(\mathbf{x}) = 0$$

Substituting $\mathbf{w}_0 = [0, 0, -\frac{1}{\sqrt{2}}, 0, 0, 0]^\top$:

$$\mathbf{w}_0^\top \phi(\mathbf{z}) = \begin{bmatrix} 0 & 0 & -\frac{1}{\sqrt{2}} & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ x_1^2 \\ \sqrt{2} x_1 x_2 \\ x_2^2 \\ \sqrt{2} x_1 \\ \sqrt{2} x_2 \end{bmatrix} = 0$$

$\Rightarrow x_1 x_2 = 0$ is the decision boundary.

Nonlinear Decision Boundary



The nonlinear decision boundary corresponds to:

$$y = -x_1x_2$$

x_1	x_2	$y = -x_1x_2$
-1	-1	-1
-1	+1	+1
+1	-1	+1
+1	+1	-1

SVM for OR Logic Function



The OR dataset is **linearly separable**. A simple separating hyperplane is $\mathbf{w}\mathbf{x} + b = 0$ where $\mathbf{w} = [1, 1]$, $b = -0.5$.

Question: Find the equation of the hyperplane that *maximizes* the margin.

i	x_1	x_2	y_i
1	0	0	-1
2	0	1	+1
3	1	0	+1
4	1	1	+1

Soft-Margin SVM for the OR Logical Function



Let $\mathbf{w} \in \mathbb{R}^2$, bias $b \in \mathbb{R}$, slack variables $\xi_i \geq 0$, and regularisation parameter $C > 0$.

Primal (soft-margin) SVM:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^4 \xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad i = 1, \dots, 4, \\ & \xi_i \geq 0, \quad i = 1, \dots, 4. \end{aligned}$$

This enforces margin constraints but allows misclassification controlled by the ξ_i and penalty C .

Hinge-Loss Function



Hinge-loss (unconstrained) form:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)).$$

Dual Formulation



Introduce dual variables α_i :

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^4 \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^{\top} \mathbf{x}_j \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, 4, \\ & \sum_{i=1}^4 \alpha_i y_i = 0. \end{aligned}$$

After solving for α , recover:

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i \quad \text{and compute } b \text{ from support vectors.}$$